

LTF — TFT 스타일 오토배틀러 (Unreal Engine 5.6, C++) | 담당: Client Programmer, 전투 상태머신 설계
파일: Source/LTF/Unit/LTFUnitBase.cpp, Source/LTF/Battle/Combat/LTFCombatTypes.h

1. 이 코드는 무엇인가

LTF는 5인 팀이 Unreal Engine 5.6으로 개발 중인 TFT(Teamfight Tactics) 스타일 오토배틀러입니다. 저는 이 중 전투 상태머신과 공격 판정 파이프라인을 설계·구현했습니다. 아래 코드는 유닛(챔피언/크립) 한 기가 전투 중 "적을 찾는다 → 다가간다 → 공격한다 → 스킬을 쓴다 → 죽거나 승리 연출을 한다"는 흐름을 매 프레임 처리하는 상태머신의 핵심 부분입니다. 이 코드는 협업 중 팀원들과 함께 다듬은 파일에서, 제가 직접 작성한 함수만 골라 담았습니다.

2. 상태 타입 정의

상태머신이 오가는 7가지 상태입니다. CastingSkill은 이후 GAS 스킬 연동을 염두에 두고 미리 정의해 둔 상태입니다.

```
// Source/LTF/Battle/Combat/LTFCombatTypes.h

// 전투 중 챔피언의 행동 상태. TickCombat 내 상태머신의 기준값
UENUM(BlueprintType)
enum class ELTFCombatState : uint8
{
    Idle          UMETA(DisplayName = "Idle"),
    Celebrating   UMETA(DisplayName = "Celebrating"),
    Searching     UMETA(DisplayName = "Searching"),
    MovingToTarget UMETA(DisplayName = "MovingToTarget"),
    Attacking     UMETA(DisplayName = "Attacking"),
    CastingSkill  UMETA(DisplayName = "CastingSkill"),
    Dead         UMETA(DisplayName = "Dead")
};
```

3. TickCombat() — 매 프레임 상태 전이

매 프레임 호출되며 switch (CombatState)로 분기합니다. 서버에서만 실행되도록 맨 위에서 막고, 클라이언트는 위치·애니메이션 복제만 받도록 설계했습니다 — 멀티플레이어 환경에서 판정 로직과 연출을 분리하기 위한 원칙을 상태머신 진입점부터 지킨 것입니다.

```

void ALTUnitBase::TickCombat(float DeltaTime)
{
    // 전투 시뮬레이션은 서버에서만 실행, 클라이언트는 위치/애니메이션 복제로 처리
    if (!HasAuthority())
    {
        return;
    }

    if (!ASC)
    {
        UE_LOG(LogTemp, Warning, TEXT("ALTUnitBase::TickCombat ASC is Null"));
        return;
    }

    // CombatManager에 등록되지 않은 유닛 (대기석 등) skip
    if (!IsInCombat) return;

    // 사망 상태이거나 죽어 상태라면 return
    FGameplayTagContainer CombatStateTags;
    CombatStateTags.AddTag(LTFGameplayTag::State_Action_Dead);
    CombatStateTags.AddTag(LTFGameplayTag::State_Action_Celebrate);

    if (ASC->HasAnyMatchingGameplayTags(CombatStateTags))
    {
        return;
    }
    const bool bIsCrowdControlled =
        ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Debuff_Stun) ||
        ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Debuff_Airborne);
    if (bIsCrowdControlled)
    {
        if (!bWasStunned) // CC 시작: 진행 중인 어빌리티/애니를 즉시 중단한다.
        {
            UE_LOG(LogTemp, Warning, TEXT("[CC][Start] %s 행동 불가 시작"), *GetName());
        }
        {
            CancelActiveCombatAbilities();
            GetCharacterMovement()->StopMovementImmediately();
            ASC->RemoveLooseGameplayTag(LTFGameplayTag::State_Movement_Walk);
            ASC->RemoveLooseGameplayTag(LTFGameplayTag::State_Movement_Idle);
            // CC 걸리기 직전 MovingToTarget 상태였다면 bIsWalking이 true로 고정된 채 남아
            // 애니메이션이 계속 걷는 것처럼 보이는 문제 방지 (특히 끌려오는 동안 눈에 땀)
            bIsWalking = false;
            bWasStunned = true;
        }
        return;
    }
    if (bWasStunned) // 기절 해제: Idle 태그 복구 후 상태 재탐색
    {
        bWasStunned = false;
        ASC->AddLooseGameplayTag(LTFGameplayTag::State_Movement_Idle);
        CombatState = ELTFCombatState::Searching;
    }

    // 대시 스킬 이동 중에는 일반 전투 루프 건너뛰기 - GA가 직접 이동을 제어
    if (ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Skill_Moving))
    {
        return;
    }

    // 타겟이 사망했으면 즉시 재탐색
    if (Target && !Target->IsAlive())
    {
        SetTarget(nullptr);
    }
}

```

```

        TransitionCombatState(ELTFCombatState::Searching);
    }

    switch (CombatState)
    {
    case ELTFCombatState::Searching:
        SetTarget(FindNearestEnemy());
        if (Target)
        {
            TransitionCombatState(IsInAttackRange() ? ELTFCombatState::Attacking : ELTFCombatState::MovingToTarget);
        }
        else
        {
            if (HasAliveEnemies())
            {
                GetCharacterMovement()->StopMovementImmediately();
            }
            else
            {
                // 적이 없으면 전투 종료 연출
                TransitionCombatState(ELTFCombatState::Celebrating);
            }
        }
    }
    break;

    case ELTFCombatState::MovingToTarget:
        if (!Target)
        {
            TransitionCombatState(ELTFCombatState::Searching);
            break;
        }

        // 0.25s마다 타겟 유효성 검사 (사망 / 사거리 진입 / 더 가까운 적)
        TargetCheckTimer += DeltaTime;
        if (TargetCheckTimer >= TargetCheckInterval)
        {
            TargetCheckTimer = 0.f;

            // 1. 타겟 사망
            if (!Target->IsAlive())
            {
                SetTarget(nullptr);
                TransitionCombatState(ELTFCombatState::Searching);
                break;
            }

            // 2. 공격 사거리 진입
            if (IsInAttackRange())
            {
                GetCharacterMovement()->StopMovementImmediately();
                InvalidatePath();
                AttackTimer = 0.f;
                TransitionCombatState(ELTFCombatState::Attacking);
                break;
            }

            // 3. 더 가까운 적이 2칸 이상 가까우면 타겟 교체 (진동 방지용 +1 마진)
            ALTFUnitBase* Nearest = FindNearestEnemy();
            if (IsValid(Nearest) && Nearest != Target)
            {
                ULTFHexGridSubsystem* Grid = nullptr;
                if (ALTFGameMode* GM = GetWorld()->GetAuthGameMode<ALTFGameMode>())
                    Grid = GetWorld()->GetSubsystem<ULTFHexGridSubsystem>() : nullptr;
            }
        }
    }
}

```

```

        if (Grid)
        {
            const int32 CurDist = Grid->GetHexDistanceByIndex(CombatBoardIndex, GetCurrentTileIndex(),
                                                            Target->GetCurrentTileIndex());

            const int32 NewDist = Grid->GetHexDistanceByIndex(CombatBoardIndex, GetCurrentTileIndex(),
                                                            Nearest->GetCurrentTileIndex());

            if (NewDist + 1 < CurDist)
            {
                SetTarget(Nearest);
                InvalidatePath();
                TransitionCombatState(ELTFCombatState::Searching);
                break;
            }
        }
    }

    MoveTowardTarget();
    break;

case ELTFCombatState::Attacking:
    if (!Target || !Target->IsAlive() || !IsInAttackRange())
    {
        TransitionCombatState((Target && Target->IsAlive())
                               ? ELTFCombatState::MovingToTarget
                               : ELTFCombatState::Searching);

        break;
    }
    // 공격 중 타겟 방향으로 즉시 회전
    {
        const FVector Dir = (Target->GetActorLocation() - GetActorLocation()).GetSafeNormal2D();
        if (!Dir.IsNearlyZero())
        {
            SetActorRotation(FMath::RInterpTo(GetActorRotation(), Dir.Rotation(), DeltaTime, 15.f));
        }
    }
    AttackTimer -= DeltaTime;
    if (AttackTimer <= 0.0f)
    {
        TryAttack();
    }
    break;

case ELTFCombatState::Celebrating:
    break;

default:
    break;
}
}

```

4. TryAttack() — 공격 트리거와 크리티컬 판정

공격 타이머가 0이 되면 호출됩니다. 이번 공격의 크리티컬 여부를 한 번만 롤(roll)해서 bPendingCrit에 저장해두는데, 이는 몽타주 선택(LTFGA_Attack)과 데미지 적용(LTFGA_AttackHitCheck)이라는 서로 다른 두 Gameplay Ability가 "같은 공격의 크리티컬 여부"를 동일하게 봐야 하기 때문입니다.

```

void ALTFUnitBase::TryAttack()
{
    if (!Target)
    {
        return;
    }
    if (ASC && ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Debuff_Stun))
    {
        return;
    }
    if (ASC && ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Debuff_Airborne))
    {
        return;
    }
    if (ASC && ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Debuff_Disarm))
    {
        return;
    }
    if (ASC && ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Action_Skill))
    {
        return;
    }

    AttackTimer = 1.0f / FMath::Max(AttributeSet->GetAttackSpeed(), 0.01f);

    // 공격 1회당 한 번만 크리티컬 롤. 몽타주 선택(LTFGA_Attack)과 데미지 적용(LTFGA_AttackHitCheck)이 같은 값을 보도록 함
    const float CritRoll = FMath::FRand();
    bPendingCrit = CritRoll < AttributeSet->GetCriticalRate();

    if (ASC)
    {
        const bool bActivated = ASC->TryActivateAbilitiesByTag(
            FGameplayTagContainer(LTFGameplayTag::Ability_Action_Attack));
    }
}

```

5. TransitionCombatState() — 상태 전이와 애니메이션 태그 동기화

상태가 바뀔 때마다 GAS 태그를 갱신해 AnimBP가 태그만 보고 걷기/대기/공격/스킬 모션을 판단할 수 있게 합니다. 코드와 애니메이션 사이의 결합을 태그 하나로 좁혀두면, 애니메이터가 상태머신 내부 구현을 몰라도 태그 조건만으로 블렌드 스페이스를 구성할 수 있습니다.

```

void ALTFUnitBase::TransitionCombatState(ELTFCombatState NewState)
{
    if (CombatState == NewState) return;
    const ELTFCombatState PrevState = CombatState;

    UE_LOG(LogTemp, Warning, TEXT("[TransitionCombatState] %s | %d -> %d"), *GetName(), (int32)CombatState,
        (int32)NewState);

    CombatState = NewState;
    bIsWalking = (NewState == ELTFCombatState::MovingToTarget);
    if (PrevState == ELTFCombatState::CastingSkill && NewState == ELTFCombatState::Attacking)
    {
        AttackTimer = 0.f;
    }

    // AnimBP가 GAS 태그로 상태를 판단하도록 태그 갱신
    // 대기석 유닛(bIsInCombat=false)은 태그를 건드리지 않음
    if (ASC && bIsInCombat)
    {
        // 이동 태그
        if (NewState == ELTFCombatState::MovingToTarget)
        {
            ASC->AddLooseGameplayTag(LTFGameplayTag::State_Movement_Walk);
            ASC->RemoveLooseGameplayTag(LTFGameplayTag::State_Movement_Idle);
        }
        else
        {
            ASC->RemoveLooseGameplayTag(LTFGameplayTag::State_Movement_Walk);
            ASC->AddLooseGameplayTag(LTFGameplayTag::State_Movement_Idle);
        }

        // 전투 액션 태그 - 진입 상태에 맞는 태그 추가, 나머지 제거
        ASC->RemoveLooseGameplayTag(LTFGameplayTag::State_Action_Attack);
        ASC->RemoveLooseGameplayTag(LTFGameplayTag::State_Action_Skill);
        if (NewState != ELTFCombatState::Dead)
        {
            ASC->RemoveLooseGameplayTag(LTFGameplayTag::State_Action_Dead);
        }

        if (NewState == ELTFCombatState::Attacking)
        {
            ASC->AddLooseGameplayTag(LTFGameplayTag::State_Action_Attack);
        }
        else if (NewState == ELTFCombatState::CastingSkill)
        {
            ASC->AddLooseGameplayTag(LTFGameplayTag::State_Action_Skill);
        }
        else if (NewState == ELTFCombatState::Dead)
        {
            if (!ASC->HasMatchingGameplayTag(LTFGameplayTag::State_Action_Dead))
            {
                ASC->AddLooseGameplayTag(LTFGameplayTag::State_Action_Dead);
            }
        }
        else if (NewState == ELTFCombatState::Celebrating)
        {
            ASC->AddLooseGameplayTag(LTFGameplayTag::State_Action_Celebrate);
            Celebrate();
        }
    }
}

```

6. 설계 근거 요약

이 상태머신을 설계할 때 지킨 원칙은 두 가지입니다.

첫째, **판정은 서버, 연출은 클라이언트**. `TickCombat` 최상단의 `HasAuthority()` 체크가 그 경계선입니다. 이 원칙을 지키지 않아서 실제로 겪은 문제가 있습니다 — 공격 애니메이션(몽타주)은 서버가 재생을 지시하면 모든 클라이언트에 복제되어 각자 재생되는데, 데미지 판정을 트리거하는 `AnimNotify`가 복제된 모든 인스턴스에서 동일하게 실행되어 서버 1개와 클라이언트 N개가 각각 판정 이벤트를 보내는 바람에 평타 데미지가 중복 적용된 적이 있었습니다. 이후로는 판정 로직 진입점마다 권한 검증을 습관적으로 넣게 되었고, 이 상태머신의 `TickCombat`도 그 습관이 반영된 코드입니다.

둘째, **상태 전이는 한 곳(`TransitionCombatState`)에서만**. 상태값을 여기저기서 직접 대입하지 않고 이 함수를 거치게 강제하면, 상태가 바뀔 때 반드시 같이 갱신되어야 하는 애니메이션 태그·타이머 초기화를 빠뜨릴 걱정이 없습니다.